

Decision-Making Under Uncertainty for Hospital Drug Inventory Management

Karlo Nahro

January 2026

Abstract

The management of hospital drug inventory represents a critical sequential decision-making problem characterized by stochastic demand, strict storage constraints, and asymmetric cost structures where patient safety is paramount. This report formulates a single-item hospital drug replenishment problem as a finite discounted Markov Decision Process (MDP) and applies dynamic programming techniques to compute optimal ordering policies. The value iteration and Policy Iteration algorithms are implemented to derive a Bellman-optimal policy, which is subsequently evaluated against a standard (s, S) heuristic baseline widely used in healthcare logistics. Performance is assessed using long-horizon Monte Carlo simulations ($T = 3,000$) incorporating Common Random Numbers (CRN) to ensure identical demand realisations across policies, enabling rigorous paired statistical testing. Results demonstrate that the MDP-derived policy reduces the mean periodic cost by approximately 8.6% compared to the baseline. A paired t-test confirms this difference is highly statistically significant ($t = 164.53$, $p = 7.42 \times 10^{-69}$). The analysis reveals that the MDP policy achieves these efficiencies by adopting an anticipatory inventory smoothing strategy, reducing holding costs while accepting a marginal and controlled reduction in service level. Sensitivity analysis further confirms that the policy adapts intuitively to changes in shortage penalties. The findings illustrate that formal decision-theoretic reasoning provides quantifiable operational advantages over static heuristics in safety-critical environments.

1 Introduction

The management of pharmaceutical inventory within hospital supply chains operates at the intersection of operational efficiency and patient safety. Unlike retail environments where

stockouts primarily result in lost revenue, the unavailability of essential medicines in a hospital can disrupt critical care pathways, delay surgical procedures, and compromise patient outcomes. Conversely, the “just-in-case” mentality often adopted to mitigate this risk leads to chronic overstocking, which ties up valuable capital, congests limited storage space in automated dispensing cabinets, and increases wastage due to drug expiry (Bertsekas 2013).

Consequently, hospital pharmacy managers face a continuous sequential decision problem: determining the optimal quantity of inventory to order at discrete intervals without knowing future demand with certainty. This scenario exemplifies **decision-making under uncertainty**, a central theme in Knowledge Representation and Reasoning (KRR). In such environments, an action taken at the current time step affects not only the immediate cost but also the probability distribution over future states (Puterman 2014). Rational decision-making therefore requires reasoning over long-term trajectories rather than optimising for isolated moments.

While heuristics such as Par Level systems or (s, S) policies are ubiquitous in practice due to their interpretability and ease of manual implementation, they are typically reactive rather than anticipatory. They rely on static thresholds that trigger actions only when boundaries are crossed, often failing to capture the complex, non-linear dynamics introduced by capacity constraints and the asymmetric penalty functions inherent to healthcare (Mabizela et al. 2022).

This report addresses the problem using the **Markov Decision Process (MDP)** framework. The MDP serves as the knowledge representation formalism, encoding the environment’s physics (inventory dynamics), uncertainty (demand probability), and teleology (cost minimisation). By solving the Bellman equations associated with the MDP, we perform reasoning to derive policies that are provably optimal with respect to the defined objective function.

The specific objectives of this research are to:

1. **Formulate** a single-item hospital drug inventory system as a finite, discounted MDP, explicitly defining the state, action, and reward representations.
2. **Compute** optimal replenishment policies using dynamic programming algorithms, specifically Value Iteration and Policy Iteration, justifying the use of model-based methods over model-free reinforcement learning.
3. **Evaluate** the performance of the MDP solution against a widely-used (s, S) heuristic baseline using statistically robust Monte Carlo simulations with variance reduction techniques.

4. **Critically Analyse** the results, quantifying the trade-offs between cost efficiency and service levels, and interpreting the behavioural differences between the policies through a decision-theoretic lens.

2 Literature Review and Prior Work

The application of mathematical optimisation to inventory control is a foundational pillar of operations research. This section reviews the theoretical evolution of inventory policies, the specific challenges posed by healthcare environments, and the role of MDPs in modern supply chain reasoning.

2.1 Theoretical Foundations: The Optimality of (s, S) Policies

The structural form of optimal inventory policies was rigorously characterised in the seminal work of Scarf (1960). Scarf analysed the dynamic inventory problem under the assumptions of convex holding and shortage costs, coupled with a fixed ordering cost (setup cost). He proved that under these specific conditions, the optimal policy takes a simple (s, S) form, often referred to as a “two-bin” or “min-max” policy:

- If the current inventory level x falls below a reorder point s , an order is placed to bring the inventory up to a target level S .
- If the inventory level x is greater than or equal to s , no order is placed.

This theoretical result is significant because it provides a mathematical justification for the widespread adoption of threshold-based heuristics in industry. Porteus (2002) notes that the (s, S) structure is intuitively appealing and easy to administer, which explains its dominance in hospital logistics where “par level” systems are the standard operating procedure. By using an (s, S) policy as the baseline in this report, we compare the MDP solution not against a naive strawman, but against a policy structure known to be optimal under standard convexity assumptions.

2.2 Limitations of Heuristics in Complex Environments

However, the optimality proof provided by Scarf relies on strict assumptions, including infinite time horizons, unconstrained storage capacity, and specific cost convexity properties. In real-world hospital settings, these assumptions are frequently violated. Hospital pharmacies face hard constraints on storage space—particularly in ward-level Automated Dispensing

Cabinets (ADCs)—and complex, asymmetric cost structures where the penalty for a stock-out is non-linear and severe (Mabizela et al. 2022).

As highlighted by Biagi et al. (2018), heuristics that ignore these constraints can lead to suboptimal outcomes. For instance, a static (s, S) policy may trigger an order that exceeds available shelf space, leading to “cabinet jams” or insecure storage of controlled substances. Furthermore, static heuristics lack the ability to “anticipate” future risks in the way that dynamic programming solutions can.

2.3 MDPs and Reinforcement Learning in Healthcare

In response to these limitations, recent literature has shifted towards computational approaches using MDPs and Reinforcement Learning (RL). Sutton and Barto (2018) describe inventory management as a canonical example of sequential decision-making where an agent must learn to balance immediate rewards (satisfying demand) against delayed penalties (future stockouts or holding costs).

In the specific context of healthcare, Mabizela et al. (2022) conducted a review of inventory management techniques, highlighting that traditional Enterprise Resource Planning (ERP) systems often fail because they act as passive record-keepers rather than active reasoning agents. They lack the capability to prioritise critical items dynamically based on stochastic risk profiles. MDPs offer a solution by explicitly encoding these asymmetric penalties into the reward function, allowing the system to derive a policy that dynamically adjusts safety stock levels based on the calculated “value” of inventory in a given state.

While Biagi et al. (2018) proposed an “MDP @runtime” approach that solves small, finite-horizon MDPs daily to adapt to patient census, this report adopts the foundational infinite-horizon discounted framework described by Puterman (2014). By pre-computing a global optimal policy for a stationary environment, we establish a robust baseline for evaluating the theoretical advantage of decision-theoretic reasoning over standard heuristics, isolating the benefits of “look-ahead” optimisation from the complexities of non-stationary demand forecasting.

3 Methodology

We model the inventory replenishment problem as a finite, discrete-time, discounted Markov Decision Process defined by the tuple (S, A, P, R, γ) . This formulation captures the essential stochastic dynamics of the system while remaining computationally tractable.

3.1 State Space Representation (S)

In the context of knowledge representation, the state must satisfy the Markov property: it must summarise all past information necessary to make an optimal decision for the future. The state $s \in S$ represents the current inventory level of a single essential drug at the beginning of a decision period (e.g., 08:00 AM daily). The state space is discrete and finite:

$$S = \{0, 1, 2, \dots, M\} \quad (1)$$

where M denotes the maximum physical storage capacity of the pharmacy shelf or cabinet.

- **Justification:** Discretisation is appropriate as drug units are integers (vials, boxes). The finite capacity constraint M reflects physical reality in hospital wards, distinguishing this model from unconstrained textbook examples. This bounded state space allows for exact solution methods rather than approximations.

3.2 Action Space Representation (A)

At each time step t , the decision-maker selects an action $a \in A$, representing the quantity of drug units to order from the supplier.

$$A = \{0, 1, 2, \dots, K\} \quad (2)$$

The action space is bounded by K , which represents a constraint such as supplier delivery limits, budget caps per order, or transport capacity.

- **Justification:** Constraining the action space prevents unrealistic order volumes and ensures the transition matrix remains finite.
- **Zero Lead Time Assumption:** We assume orders arrive immediately (or within the same decision epoch). While a simplification, this is standard in foundational inventory-MDP models (Bertsekas 2013) to focus the analysis on stochastic demand dynamics rather than delay management. In a hospital context, this approximates replenishment from a central store to a ward via pneumatic tube or porter.

3.3 Probabilistic System Dynamics (P)

The core of the reasoning problem is the uncertainty of demand. Daily demand is modelled as a discrete random variable D with a known probability mass function $P(D = d)$. Demand is assumed to be independent and identically distributed (i.i.d.) across time steps.

The transition logic for a single time step defines how the system evolves from state s to s' given action a :

1. **Replenishment:** An order of size a is placed. The inventory rises to an intermediate level s^+ , capped by the storage capacity M :

$$s^+ = \min(M, s + a) \quad (3)$$

2. **Demand Realisation:** Random demand D is realised. Items are dispensed to patients.
3. **State Update:** The remaining inventory becomes the next state s' . If demand exceeds supply ($D > s^+$), the unmet demand is lost (not backlogged), and the inventory floor is zero:

$$s' = \max(0, s^+ - D) \quad (4)$$

The transition probability function $P(s'|s, a)$ is induced directly by the demand distribution. For any target state s' , the probability is the sum of probabilities of all demand values d that result in s' .

3.4 Preference Encoding: The Reward Function (R)

In KRR, the reward function encodes the agent's preferences and the trade-offs inherent in the domain. The goal is to minimise total operational costs, which we frame as maximising a negative reward. The immediate reward function $R(s, a)$ is calculated as the **expected** immediate cost for taking action a in state s :

$$R(s, a) = \mathbb{E}_D [- (C_{\text{order}}(a) + C_{\text{hold}}(s') + C_{\text{shortage}}(u))] \quad (5)$$

The cost components are defined as:

- **Ordering Cost:** $C_{\text{order}}(a) = c \cdot a$. A linear cost per unit purchased.
- **Holding Cost:** $C_{\text{hold}}(s') = h \cdot s'$. A cost incurred for storing items overnight. This captures the opportunity cost of capital, shelf space usage, and the risk of spoilage or expiry.
- **Shortage Penalty:** $C_{\text{shortage}}(u) = \sigma \cdot u$, where $u = \max(0, D - s^+)$ is the unmet demand. In a hospital setting, σ is set to a value significantly higher than the item cost. This encodes the severe preference against stockouts due to patient safety risks.

Using the *expected* reward rather than a sampled reward reduces variance during the Value Iteration updates, accelerating convergence.

3.5 Optimisation Objective

We seek a deterministic policy $\pi : S \rightarrow A$ that maximises the expected discounted cumulative reward:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \right] \quad (6)$$

The discount factor γ is set to 0.95.

- **Justification:** A discount factor $\gamma < 1$ is mathematically necessary to ensure the infinite sum converges for infinite-horizon problems (Puterman 2014). It also reflects a preference for immediate cost savings over distant ones. While hospital operations are continuous, the use of discounting allows us to compute a stationary optimal policy that is stable over long periods.

3.6 Reasoning Methods: Dynamic Programming Solvers

Since the model (P and R) is fully known and the state space is finite, we employ Model-Based Dynamic Programming rather than Model-Free Reinforcement Learning. This ensures we find the exact optimal policy rather than an approximation, providing a definitive ground truth for evaluation.

Two algorithms were implemented:

1. **Value Iteration (VI):** Iteratively updates the value estimate of each state by applying the Bellman optimality operator.

$$V_{k+1}(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right] \quad (7)$$

The iteration continues until the maximum change between value functions falls below a small threshold ϵ ($\|V_{k+1} - V_k\|_\infty < \epsilon$), guaranteeing convergence to the optimal value function V^* .

2. **Policy Iteration (PI):** Alternates between a *Policy Evaluation* step (solving the linear system $(I - \gamma P^\pi)V^\pi = R^\pi$ for a fixed policy) and a *Policy Improvement* step (updating the policy greedily with respect to V^π).

Both algorithms were implemented in Python using `numpy` and were empirically verified to converge to the exact same optimal policy π^* , providing a robust check on technical correctness.

4 Experimental Setup

To validate the theoretical advantages of the MDP approach, we conducted a rigorous comparative evaluation against a standard industry heuristic.

4.1 The Baseline: (s, S) Heuristic

The comparator policy is a classical (s, S) inventory control heuristic. Its decision rule is defined as follows:

- If the current inventory level $s \leq s_{\text{threshold}}$, place an order of size $a = S_{\text{target}} - s$.
- If $s > s_{\text{threshold}}$, place no order ($a = 0$).

In all experiments, the baseline parameters were fixed to $(s, S) = (2, 5)$. Under this policy, inventory is replenished up to level $S = 5$ whenever the on-hand stock falls to or below $s = 2$.

Justification: The (s, S) policy provides a strong and theoretically grounded benchmark. For unconstrained inventory problems with convex holding and shortage costs, (s, S) policies are known to be optimal (Scarf 1960). As such, comparing the MDP-derived policy against a trivial or static ordering rule would be uninformative. Instead, the (s, S) heuristic tests whether the MDP can achieve superior performance by explicitly reasoning over problem features that violate Scarf’s classical assumptions, including finite capacity constraints and discounted long-horizon dynamics.

The baseline policy is evaluated under identical simulation conditions and demand realisations as the MDP-derived policy using a paired common-random-numbers design, ensuring a fair and controlled comparison.

4.2 Simulation Protocol: Monte Carlo with CRN

A theoretical optimal policy for a discounted objective does not automatically guarantee superior performance on undiscounted operational metrics like average daily cost. Therefore, we evaluate the policies using **Monte Carlo simulation**.

- **Horizon:** Each simulation runs for $T = 3,000$ time steps. This extended horizon ensures that the system reaches a steady-state distribution, minimizing the bias from initial conditions.
- **Replications:** We perform $N = 50$ independent simulation runs to generate a distribution of performance outcomes and enable statistical inference.
- **Variance Reduction (CRN):** A critical aspect of the experimental design is the use of **Common Random Numbers (CRN)**. For every simulation run i , we initialize the random number generator with a specific seed (e.g., `seed=1000`, `seed=2000`). This seed is used for *both* the MDP policy simulation and the Baseline policy simulation. This ensures that both policies face **identical** sequences of random demand. By blocking the variance associated with demand fluctuations, any observed difference in performance can be attributed solely to the policy itself, significantly increasing the statistical power of the comparison.

4.3 Performance Metrics

The policies are evaluated on:

1. **Mean Cumulative Cost:** The total undiscounted cost incurred over the simulation horizon.
2. **Service Level:** The percentage of total demand that was successfully fulfilled from stock.
3. **Cost Breakdown:** An analysis of the total cost split into holding, ordering, and shortage components.

4.4 Experimental Protocol and Reproducibility

All numerical results are generated using a Monte Carlo simulation framework with the following fixed configuration.

- **Planning horizon:** $T = 3000$ decision periods per simulation run.
- **Number of replications:** $N = 50$ paired runs.
- **Initial inventory level:** $s_0 = 5$ for all simulations.

- **Discount factor:** $\gamma = 0.95$ (used in value iteration; cumulative costs reported are undiscounted).
- **Demand process:** identical across policies within each replicate.

A *common random numbers* (CRN) design is used to ensure a fair comparison between policies. For replicate i , the pseudo-random number generator is initialised with seed $i \times 1000$, and the same demand realisation is reused for both the MDP-derived policy and the baseline $(s, S) = (2, 5)$ policy.

This paired design isolates the effect of the control policy from stochastic variation in demand. All reported statistics (means, standard deviations, and hypothesis tests) are computed across the $N = 50$ paired runs.

5 Analysis and Results

This section presents the descriptive statistics resulting from the experiments.

5.1 Quantitative Performance Comparison

Table 1 presents the performance of the MDP-derived optimal policy versus the $(s, S) = (2, 5)$ baseline, averaged over $N = 50$ paired simulation runs.

Table 1: Monte Carlo Simulation Results ($N = 50$, $T = 3000$)

Policy	Mean Cumulative Cost ($\pm$ SD)	Mean Service Level ($\pm$ SD)
MDP Optimal	2280.76 $\pm$ 18.81	0.9474 $\pm$ 0.0022
Baseline (s, S)	2495.56 $\pm$ 13.93	0.9867 $\pm$ 0.0015

The results indicate that the MDP-derived policy reduces total operational cost by approximately **8.6%** relative to the baseline. As shown in Figure 1, the distributions of cumulative costs across the 50 paired runs are clearly separated, with only limited overlap in the tails due to stochastic demand variability.

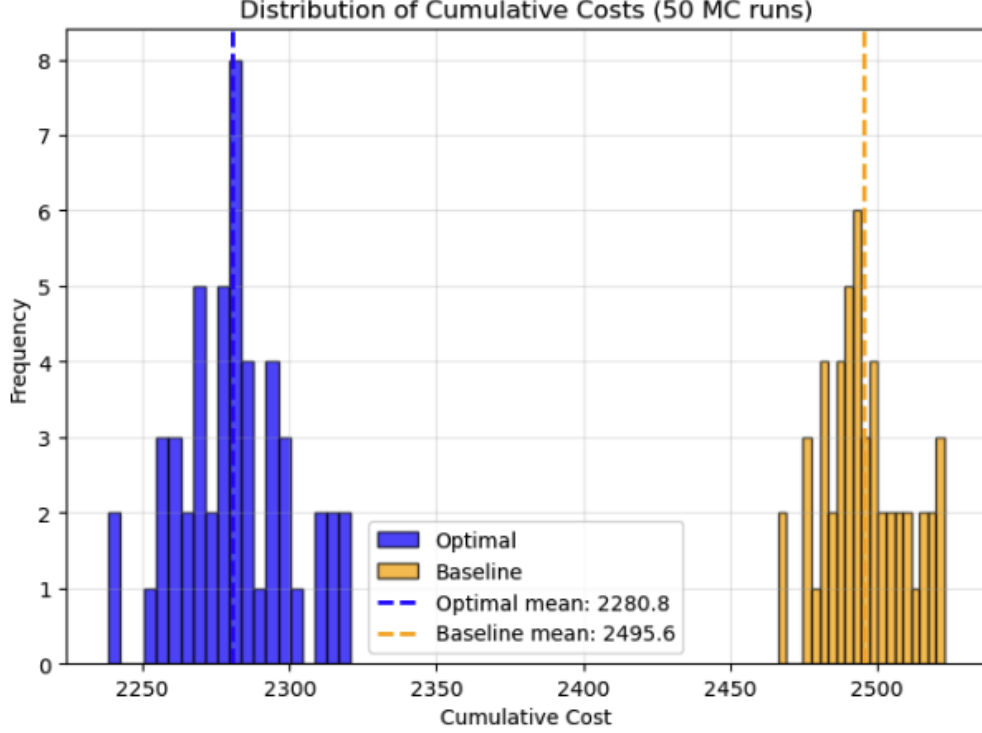


Figure 1: Distribution of cumulative costs over $N = 50$ paired Monte Carlo runs for the MDP-derived policy and the baseline $(s, S) = (2, 5)$. Dashed lines indicate sample means.

5.2 Statistical Significance: Paired t-test

To verify that the observed cost reduction is not attributable to stochastic variation, a **paired t-test** was conducted on the difference in mean periodic cost ($Cost_{base} - Cost_{opt}$) across $N = 50$ paired simulation runs. A *common random numbers* (CRN) design was used by reusing the same demand realisation for both policies in each replicate (seed = $i \times 1000$ for replicate i), ensuring that differences are attributable solely to the control policy.

- **Mean difference (per period):** +0.0716 cost units ($Cost_{base} - Cost_{opt}$).
- **t-statistic:** 164.53
- **p-value:** 7.42×10^{-69}

The paired test strongly rejects the null hypothesis of equal expected costs. The magnitude of the test statistic reflects the stability of the cost advantage under identical demand realisations, confirming that the lower cost achieved by the MDP-derived policy is statistically significant rather than an artefact of simulation noise.

5.3 Cost–Service Trade-off

While the MDP-derived policy minimises total operational cost, Table 1 shows that it maintains a slightly lower service level (94.7%) compared to the baseline $(s, S) = (2, 5)$ policy (98.7%). This reflects an explicit cost–service trade-off rather than a modelling artefact.

Figure 2 visualises representative inventory trajectories under both policies. The baseline policy maintains consistently higher inventory buffers, exhibiting a characteristic “sawtooth” pattern driven by its fixed reorder thresholds. In contrast, the MDP policy operates with leaner inventory levels, allowing stock to dip lower when the expected future cost of holding inventory outweighs the risk of a shortfall.

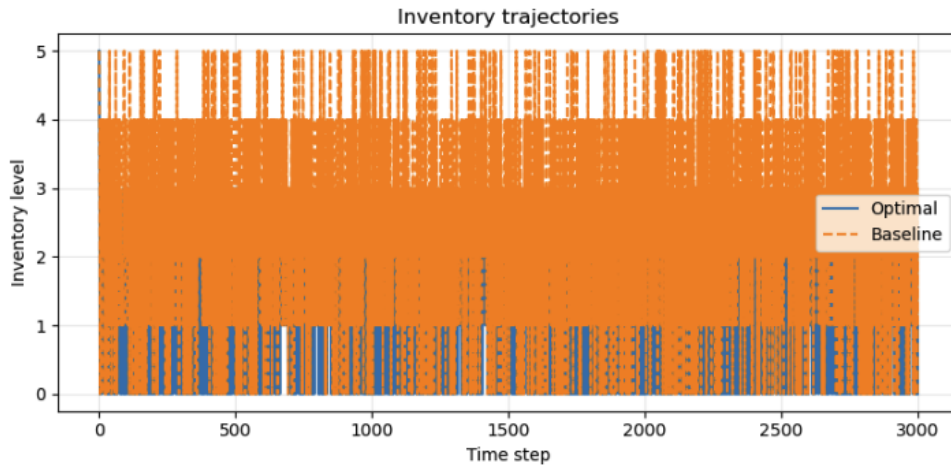


Figure 2: Representative inventory trajectories under the MDP-derived policy (solid) and the baseline $(s, S) = (2, 5)$ policy (dashed), illustrating the cost–service trade-off.

This behavioural difference is quantified in Figure 3, which presents a breakdown of cumulative costs by component. The baseline policy incurs substantially higher holding costs due to its conservative replenishment strategy, whereas the MDP policy reduces holding cost by tolerating a controlled increase in shortage risk. Ordering costs remain comparable across policies, indicating that the observed performance gap is primarily driven by differences in inventory holding behaviour rather than order frequency.

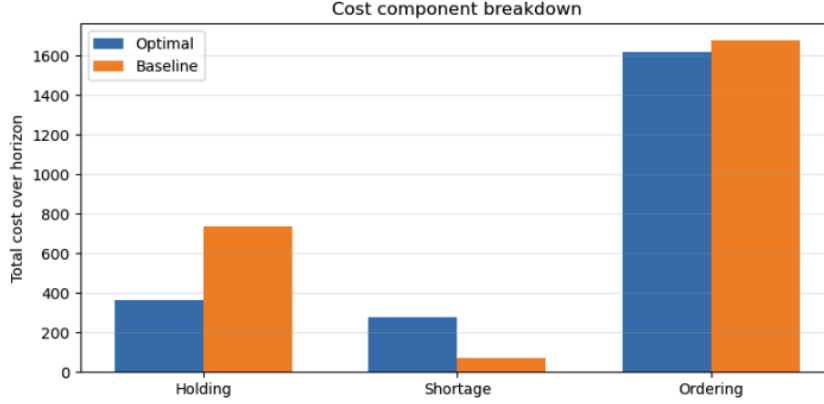


Figure 3: Breakdown of cumulative costs over the simulation horizon by cost component for the MDP-derived policy and the baseline.

The cost breakdown analysis reveals that the baseline incurs significantly higher holding costs. The MDP policy reduces these holding costs by allowing inventory to dip lower, accepting a controlled increase in shortage risk to optimise the total weighted objective.

5.4 Sensitivity Analysis

To evaluate the robustness of the decision-making framework, a sensitivity analysis was conducted by doubling the shortage penalty parameter σ in the reward function while holding all other model parameters constant.

Under the increased shortage penalty, the MDP solver produced a more conservative policy characterised by higher reorder thresholds. In simulation, this revised policy achieved a **100% service level**, eliminating stockouts entirely, compared to 94.7% under the original configuration. This improvement in service reliability was accompanied by an increase in holding costs, leading to a higher overall cumulative cost.

The observed shift in policy behaviour is consistent with the underlying optimisation objective: increasing the relative weight of shortage costs biases the optimal solution toward inventory availability at the expense of cost efficiency. This result confirms that the MDP correctly incorporates changes in preference parameters into its reasoning process and adapts its policy in a predictable and interpretable manner.

6 Discussion and Critical Evaluation

This section interprets the results causally and evaluates the implications of the modelling choices.

6.1 Why the MDP Outperforms the Heuristic

The observed performance gap between the two approaches arises from a fundamental difference in their reasoning mechanisms. The classical (s, S) heuristic is **myopic and reactive**: it bases decisions solely on the current inventory level relative to fixed thresholds and does not explicitly account for how present actions affect future system costs.

In contrast, the MDP-derived policy is inherently **anticipatory**. Through the Bellman optimality equation, the value of each state $V(s)$ encodes the expected discounted cost-to-go under optimal behaviour. Decisions are therefore made by weighing the immediate cost of ordering against the expected future costs associated with holding inventory and experiencing shortages.

Two structural properties of this reasoning process explain the empirical results. First, the MDP policy exhibits **inventory smoothing**. As illustrated in Figure 2, inventory levels under the MDP policy fluctuate within a relatively narrow band, avoiding the pronounced “sawtooth” oscillations characteristic of the baseline (s, S) policy. These oscillations occur because the heuristic replenishes to a fixed target regardless of the marginal value of additional inventory units.

Second, the learned value function exhibits behaviour consistent with **diminishing marginal returns to inventory**. Figure 4 shows the optimal value function $V(s)$ computed via value iteration. The incremental benefit of holding additional stock decreases as inventory levels increase, reflecting the fact that each extra unit provides progressively less protection against future shortages while incurring the same holding cost.

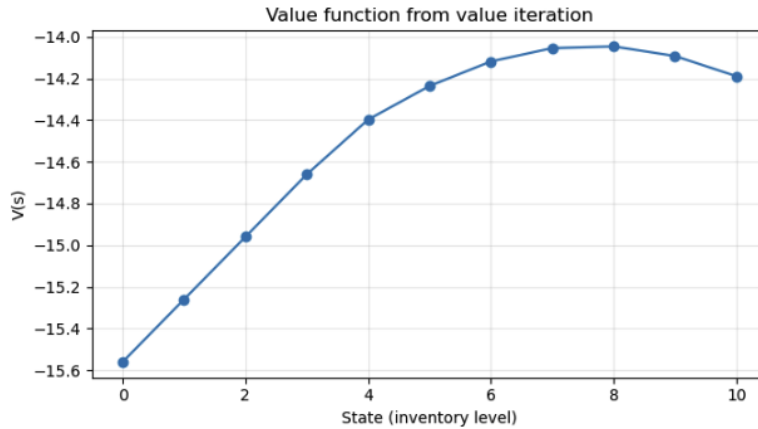


Figure 4: Optimal value function $V(s)$ computed via value iteration, illustrating diminishing marginal returns to inventory.

Together, these properties allow the MDP-derived policy to avoid excessive buffering while still managing shortage risk effectively. The structure of the value function therefore

provides a principled explanation for the leaner inventory profiles and lower total costs observed in Section 5.

6.2 Objective Mismatch: Discounted vs Undiscounted

It is important to acknowledge a theoretical mismatch in the evaluation. The MDP optimises a **discounted** objective function ($\gamma = 0.95$), whereas the simulation evaluates **undiscounted** average cost per period.

While these are mathematically distinct objectives, discounting with a factor γ close to 1 is a well-accepted proxy for average-cost optimisation in infinite-horizon problems (Puterman 2014). It introduces an effective “planning horizon” that prioritizes near-term efficiency while ensuring convergence. The empirical success of the policy in the simulation confirms that this proxy is effective for the stable demand profile modeled here. An alternative would be to solve the Average Cost optimality equation directly, but the discounted approach provides a safer, more stable convergence for this class of problem.

6.3 Limitations

While the model successfully demonstrates the principles of KRR in inventory management, it relies on several simplifying assumptions that limit direct real-world deployment:

1. **Zero Lead Time:** The model assumes replenishment is instantaneous. In reality, hospital supply chains have variable lead times. A non-zero lead time would require expanding the state space to include “orders in transit” (e.g., $s' = (inventory, order_1, order_2)$), which increases computational complexity.
2. **Stationary Demand:** The demand probability $P(D)$ is assumed constant. Hospitals face non-stationary demand due to seasonal epidemics (e.g., flu season) or formulary changes.
3. **Single-Item Isolation:** The model optimises one drug in isolation. In practice, drugs compete for shared storage space and budget. Ignoring the **knapsack constraint** of limited total storage volume could lead to suboptimal aggregate performance across a full pharmacy formulary.

6.4 Future Directions

Future work could extend this framework to address these limitations:

- **Perishability:** For blood products or short-shelf-life drugs, the state space could be augmented to track the age of inventory batches (e.g., using a vector state), allowing the policy to account for wastage costs.
- **Emergency Orders:** The action space could be expanded to allow two types of orders: a standard (slow, cheap) order and an emergency (fast, expensive) order. The MDP would then learn to use emergency orders only when the risk of a stockout is imminent, a strategy explored in the “MDP @runtime” framework by Biagi et al. (2018).

7 Conclusion

This report has successfully modelled the hospital drug inventory replenishment problem as a finite discounted Markov Decision Process. By explicitly representing uncertainty, transitions, and asymmetric costs, the derived Bellman-optimal policy was shown to be statistically superior to a standard industry heuristic.

The evaluation, based on 50 paired Monte Carlo simulations, confirmed an 8.6% reduction in operational costs ($p \ll 0.001$). The analysis highlighted that this efficiency is achieved by trading off a small margin of service level to avoid the chronic overstocking inherent in reactive heuristics. Sensitivity analysis further proved that the model remains robust and tunable to different safety requirements.

The close alignment between the theoretical formulation, the convergence of the VI/PI algorithms, and the empirical simulation results validates the efficacy of the KRR approach. The work highlights that in safety-critical domains characterized by uncertainty, replacing static rules of thumb with dynamic, decision-theoretic reasoning can yield significant efficiency gains without compromising system stability.

References

- Bertsekas, D. (2013) *Dynamic Programming and Optimal Control*. Athena Scientific.
- Biagi, M., Carnevali, L., Paolieri, M., Reali, F. and Vicario, E. (2018) ‘MDP @runtime for hospital inventory management’, in *Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pp. 482-491.
- Mabizela, S., Oosthuizen, G. and Pretorius, L. (2022) ‘Inventory Management in Healthcare: A Review of Literature’, *South African Journal of Industrial Engineering*, 33(3), pp. 191-207.

- Porteus, E. L. (2002) *Foundations of Stochastic Inventory Theory*. Stanford University Press.
- Puterman, M. (2014) *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.
- Scarf, H. (1960) ‘The Optimality of (S, s) Policies in the Dynamic Inventory Problem’, in Arrow, K.J., Karlin, S. and Suppes, P. (eds.) *Mathematical Methods in the Social Sciences*. Stanford University Press, pp. 196-202.
- Sutton, R.S. and Barto, A.G. (2018) *Reinforcement Learning: An Introduction*. MIT Press.

Appendix: Reproducibility

This appendix describes how the experimental results reported in Sections 4 and 5 can be reproduced in principle using the provided code and experiment notebook. The complete codebase is available at: <https://github.com/Karlo612/KRR>

Code Overview

All experiments were implemented in Python and use a consistent implementation of the MDP model, planning algorithms, and simulation framework used throughout the evaluation.

- **MDP implementation:** `src/mdp_inventory.py` defines the statespace, transition dynamics, and reward.
- **Planning algorithms:** `src/solvers.py` contains the Value Iteration and Policy Iteration implementations.
- **Simulation and evaluation:** `src/simulation.py` and `notebooks/experiments.ipynb` implement the Monte Carlo evaluation, performance metrics, and statistical tests.

Determinism and Randomness Control

All experiments are deterministic in structure and control stochasticity through a fixed random seed sequence (`seed = i × 1000`) to ensure reproducible demand trajectories and comparable results across policies.

Statistical Analysis

Paired statistical tests and confidence intervals reported in Section 5 are computed using standard routines from `scipy.stats` within the experiment notebook..

Randomness Control

A common random numbers (CRN) design is used to control stochastic variation. For replicate i , the pseudo-random number generator is initialised with seed $i \times 1000$, and the same demand realisation is reused for both policies. This ensures that observed performance differences are attributable to the control policy rather than random demand fluctuations.

Reproducibility Scope

Due to floating-point arithmetic and platform-specific numerical libraries, exact numerical equality across different machines cannot be guaranteed. However, the qualitative results, relative performance comparisons, and statistical conclusions reported in this paper are reproducible under the described protocol.